

MÓDULO 6 — REDES EN LINUX

6.1. Comandos de red fundamentales

Linux utiliza herramientas muy potentes para la administración de red.

- ♦ **Mostrar interfaces de red**
 - `ip a`
- ♦ **Ver rutas**
 - `ip r`
- ♦ **Ping (conectividad)**
 - `ping 8.8.8.8`
- ♦ **Resolver nombres (DNS)**
 - `nslookup google.com`
 - `dig google.com`
- ♦ **Mostrar conexiones activas**
 - `ss -tunap`
- ♦ **Trazar rutas**
 - `traceroute google.com`

6.2. Configuración de red con Netplan (Ubuntu Server)

Netplan usa archivos YAML en:

- `/etc/netplan/*.yaml`

Ejemplo IP estática:

```
network:
  version: 2
  ethernets:
    enp0s3:
```

```
dhcp4: no
addresses: [192.168.1.50/24]
gateway4: 192.168.1.1
nameservers:
  addresses: [8.8.8.8,1.1.1.1]
```

Aplicar cambios:

- `sudo netplan apply`

6.3. Configuración con NetworkManager (Ubuntu Desktop)

NetworkManager permite:

- DHCP
- IP estática
- DNS
- Conexiones VPN
- WiFi

Herramienta CLI:

- `nmcli device status`
- `nmcli connection show`

6.4. Puertos, sockets y conexiones activas

Ver puertos:

- `ss -tulnp`

Significado de flags:

- `t` → TCP
- `u` → UDP
- `l` → listening
- `n` → no resolver nombres
- `p` → mostrar procesos

6.5. Servidor SSH

Instalar:

- `sudo apt install openssh-server`

Comprobar estado:

- `systemctl status ssh`

Archivo de configuración:

- `/etc/ssh/sshd_config`

Cambiar puerto:

- Port 2222

Reiniciar:

- `sudo systemctl restart ssh`
-

6.6. Firewall UFW

Activar UFW:

- `sudo ufw enable`

Permitir SSH:

- `sudo ufw allow 22/tcp`

Denegar tráfico:

- `sudo ufw deny 80/tcp`

Ver reglas:

- `sudo ufw status numbered`
-

6.7. Diagnóstico de fallos de red

Pruebas:

1. Ping a localhost:
2. `ping 127.0.0.1`
3. Ping a tu IP.

4. Ping al gateway.
5. Ping a un DNS (8.8.8.8).
6. Ping a un dominio.

Interpretación:

Funciona	Falla	Significa
127.0.0.1	—	fallo grave de TCP/IP
IP propia	—	fallo de driver o NIC
Gateway	—	fallo de red local
8.8.8.8	—	fallo de salida a Internet
DNS	dominio	fallo de resolución DNS

30 EJERCICIOS PRÁCTICOS DEL MÓDULO 6

✓ BÁSICOS (1–10)

1. Mostrar interfaces de red

- ip a

```
(kali㉿kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:d1:f8:5d brd ff:ff:ff:ff:ff:ff
   inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
       valid_lft 83756sec preferred_lft 83756sec
   inet6 fd17:625c:f037:2:8e8a:a9f1:b99a:7bb6/64 scope global dynamic noprefixroute
       valid_lft 86339sec preferred_lft 14339sec
   inet6 fe80::e91:d06b:115:7c7a/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

2. Mostrar la ruta por defecto

- ip r

```
(kali㉿kali)-[~]
$ ip r
default via 10.0.2.2 dev eth0 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev eth0 proto kernel scope link src 10.0.2.15 metric 100
```

3. Hacer ping a tu puerta de enlace

Ejemplo:

- ping 192.168.1.1

```
(kali㉿kali)-[~]
$ ping 192.168.0.1
PING 192.168.0.1 (192.168.0.1) 56(84) bytes of data.
64 bytes from 192.168.0.1: icmp_seq=1 ttl=59 time=2.07 ms
64 bytes from 192.168.0.1: icmp_seq=2 ttl=59 time=2.62 ms
64 bytes from 192.168.0.1: icmp_seq=3 ttl=59 time=1.68 ms
64 bytes from 192.168.0.1: icmp_seq=4 ttl=59 time=2.08 ms
^C
— 192.168.0.1 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3009ms
rtt min/avg/max/mdev = 1.679/2.111/2.617/0.333 ms
```

4. Consultar DNS con nslookup

- nslookup google.com

```
(kali㉿kali)-[~]
$ nslookup google.com
Server:      80.58.61.254
Address:     80.58.61.254#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.200.110
Name:   google.com
Address: 2a00:1450:4006:817::200e
```

5. Ver conexiones activas

- ss -tunap

```
(kali㉿kali)-[~]
$ ss -tunap
Netid      State      Recv-Q     Send-Q     Local Address:Port      Peer Address:Port      Process
tcp        ESTAB      0           0           192.168.0.110:68        192.168.0.1:67
tcp        LISTEN     0           128          0.0.0.0:22             0.0.0.0:22
tcp        LISTEN     0           128          [::]:22                [::]:22
```

6. Usar traceroute

- traceroute 8.8.8.8

```
(kali㉿kali)-[~]
$ traceroute 8.8.8.8
traceroute to 8.8.8.8 (8.8.8.8), 30 hops max, 60 byte packets
1  _gateway (192.168.0.1)  0.959 ms  0.792 ms  1.091 ms
2  * * *
3  * * *
4  33.red-81-45-103.staticip.rima-tde.net (81.45.103.33)  13.621 ms  13.309 ms  13.937 ms
5  * * *
6  94.red-81-46-8.customer.static.ccg.telefonica.net (81.46.8.94)  16.367 ms  15.079 ms  23.018 ms
7  176.52.253.93 (176.52.253.93)  14.473 ms  15.153 ms  15.071 ms
8  81.173.106.55 (81.173.106.55)  15.159 ms  google-be4-grcnadno1.net.telefonicaaglobalsolutions.com (213.140.50.43)  14.479 ms  81.173.106.55 (81.173.106.55)  14.680 ms
9  192.178.110.71 (192.178.110.71)  14.254 ms  108.170.252.253 (108.170.252.253)  15.665 ms  108.170.249.11 (108.170.249.11)  16.280 ms
10  142.251.51.143 (142.251.51.143)  14.488 ms  74.125.253.197 (74.125.253.197)  15.983 ms  142.250.46.165 (142.250.46.165)  16.274 ms
11  dns.google (8.8.8.8)  15.539 ms  15.096 ms  15.128 ms
```

7. Instalar net-tools y usar ifconfig (opcional)

- sudo apt install net-tools
- ifconfig

```
(kali@kali)-[~]
└─$ sudo apt install net-tools
[sudo] password for kali:
The following packages were automatically installed and are no longer required:
  libburay2 libdata-common libgeos3.13.1 libjs-underscore libplacebo349 libqt5ct-common1.8 libsigsegv2 libsoup2.4-common libtheoradec1 libudfread0 python3-packaging-whl
  libgdal36 libgdal22 libhdf4-0-alt libogdi4.1 libportmidi0 libframe1 libsoup-2.4-1 libtheora0 libtheoraenc1 libvpx9 python3-wheel-whl
Use 'sudo apt autoremove' to remove them.

Upgrading:
  net-tools

Summary:
Upgrading: 1, Installing: 0, Removing: 0, Not Upgrading: 539
Download size: 194 kB
Freed space: 72.7 kB

Get:1 http://kali.download/kali kali-rolling/main amd64 net-tools amd64 2.10-2 [194 kB]
Fetched 194 kB in 22s (8,791 B/s)
(Reading database ... 469059 files and directories currently installed.)
Preparing to unpack .../net-tools_2.10-2_amd64.deb ...
Unpacking net-tools (2.10-2) over (2.10-1.3) ...
Setting up net-tools (2.10-2) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for kali-menu (2025.4.0) ...

(kali@kali)-[~]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.0.110 netmask 255.255.254.0 broadcast 192.168.1.255
    inet6 fe80::e91:d06b:115:7c7a prefixlen 64 scopeid 0<20<link>
    ether 08:00:27:d1:f8:5d txqueuelen 1000 (Ethernet)
    RX packets 3935 bytes 2224437 (2.1 MiB)
    RX errors 140 dropped 6 overruns 0 frame 140
    TX packets 464 bytes 38279 (37.3 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 18 bytes 1392 (1.3 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 18 bytes 1392 (1.3 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

8. Ver tu hostname

- hostname

```
(kali@kali)-[~]
└─$ hostname
kali
```

9. Modificar temporalmente tu IP

- sudo ip a add 192.168.1.200/24 dev enp0s3

```
(kali@kali)-[~]
└─$ sudo ip a add 192.168.0.200/24 dev eth0

(kali@kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d1:f8:5d brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.110/23 brd 192.168.1.255 scope global dynamic noprefixroute eth0
        valid_lft 42809sec preferred_lft 42809sec
    inet 192.168.0.200/24 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::e91:d06b:115:7c7a/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

10. Quitar una IP añadida

- `sudo ip a del 192.168.1.200/24 dev enp0s3`

```
(kali㉿kali)-[~]
$ sudo ip a del 192.168.0.200/24 dev eth0

(kali㉿kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d1:f8:5d brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.110/23 brd 192.168.1.255 scope global dynamic noprefixroute eth0
        valid_lft 42780sec preferred_lft 42780sec
    inet6 fe80::e91:d06b:115:7c7a/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

11. Repite el ejercicio anterior con otra ip

✓ INTERMEDIOS (12–20)

12. Aplicar la configuración

- `sudo netplan apply`
-

13. Ver estado de NetworkManager (Desktop)

- `nmcli device status`

```
(kali㉿kali)-[~]
$ nmcli device status
DEVICE   TYPE      STATE              CONNECTION
eth0     ethernet  connected          Wired connection 1
lo       loopback  connected (externally) lo
```

14. Revisar logs del servicio de red

- `journalctl -u NetworkManager`

```
(kali@kali)-[~]
└─$ journalctl -u NetworkManager.service
Sep 19 04:27:12 kali systemd[1]: Starting NetworkManager.service - Network Manager...
Sep 19 04:27:12 kali NetworkManager[815]: <info> [1758270432.8194] NetworkManager (version 1.52.0) is starting...
Sep 19 04:27:12 kali NetworkManager[815]: <info> [1758270432.8194] Read config: /etc/NetworkManager/NetworkManager.conf
Sep 19 04:27:12 kali NetworkManager[815]: <info> [1758270432.8400] manager[0x563422584b30]: monitoring kernel firmware
Sep 19 04:27:12 kali NetworkManager[815]: <info> [1758270432.8400] monitoring ifupdown state file '/run/network/ifupdown'
Sep 19 04:27:13 kali NetworkManager[815]: <info> [1758270433.0220] hostname: hostname: using hostnamed
Sep 19 04:27:13 kali NetworkManager[815]: <info> [1758270433.0220] hostname: static hostname changed from (none) to kali
```

15. Escanear redes WiFi disponibles

- nmcli device wifi list

16. Hacer un test de velocidad simple

- sudo apt install speedtest-cli
- speedtest

```
(kali@kali)-[~]
└─$ sudo apt install speedtest-cli
The following packages were automatically installed and are no longer required:
libbluray2 libgdata-common libgeos3.13.1 libjs-underscore libplacebo349 libqt5ct-common1.8 libsigsegv2 libsoup2.4-common libtheoradec1 libudfread0 python3-packaging-whl
libgdata36 libgdata22 libhdf4-0-alt liblog4.1 libportmidi0 libstraw1 libtheora0 libtheoraenc1 libvpx9 python3-wheel-whl
Use 'sudo apt autoremove' to remove them.

Installing:
speedtest-cli

Summary:
Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 539
Download size: 23.8 kB
Space needed: 104 kB / 17.3 GB available

Get:1 http://mirror.es.cdn-perfprod.com/kali kali-rolling/main amd64 speedtest-cli all 2.1.3-3 [23.8 kB]
Fetched 23.8 kB in 1s (24.5 kB/s)
Selecting previously unselected package speedtest-cli.
(Reading database ... 469039 files and directories currently installed.)
Preparing to unpack .../speedtest-cli_2.1.3-3_all.deb ...
Unpacking speedtest-cli (2.1.3-3) ...
Setting up speedtest-cli (2.1.3-3) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for kali-menu (2025.4.0) ...

(kali@kali)-[~]
└─$ speedtest
Retrieving speedtest.net configuration...
Testing from Telefonica de Espana Static IP (2.136.142.51)...
Retrieving speedtest.net server list...
Selecting best server based on ping...
Hosted by Orange (Madrid) [136.09 km]: 31.636 ms
Testing download speed.....
Download: 2.44 Mbit/s
Testing upload speed.....
Upload: 336.69 Mbit/s
```

17. Abrir puerto para un servicio

- sudo ufw allow 8080/tcp

```
(kali@kali)-[~]
└─$ sudo ufw allow 8080/tcp
Rule added
Rule added (v6)
```

18. Bloquear puerto

- sudo ufw deny 23/tcp


```
(kali㉿kali)-[~]  
$ sudo ufw deny 23/tcp  
Rule added  
Rule added (v6)
```

19. Ver reglas activas de firewall

- `sudo ufw status verbose`

```
(kali㉿kali)-[~]
$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN 192.168.1.40
2222/tcp ALLOW IN Anywhere
8080/tcp ALLOW IN Anywhere
23/tcp DENY IN Anywhere
2222/tcp (v6) ALLOW IN Anywhere (v6)
8080/tcp (v6) ALLOW IN Anywhere (v6)
23/tcp (v6) DENY IN Anywhere (v6)
```

20. Conectarte por SSH desde otra máquina

- `ssh usuario@IP`

✓ AVANZADOS (21–30)

21. Cambiar el puerto por defecto de SSH

Editar:

- `sudo nano /etc/ssh/sshd_config`

```
GNU nano 8.6 /etc/ssh/sshd_config *
# This is the sshd server system-wide configuration file. See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/local/bin:/usr/bin:/bin:/usr/games

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented. Uncommented options override the
# default value.

Include /etc/ssh/sshd_config.d/*.conf

Port 2222
```

```
(kali㉿kali)-[~]
$ sudo ufw allow 2222/tcp
Skipping adding existing rule
Skipping adding existing rule (v6)

(kali㉿kali)-[~]
$ sudo ufw reload
Firewall reloaded
```

22. Comprobar que SSH escucha en el nuevo puerto

- `ss -tulnp | grep ssh`

```
(kali@kali)-[~]
$ sudo ss -tulnp | grep 22
tcp    LISTEN 0      128          0.0.0.0:2222  0.0.0.0:*    users:(("sshd",pid=3369,fd=6))
tcp    LISTEN 0      128          [::]:2222   [::]:*       users:(("sshd",pid=3369,fd=7))
```

23. Forzar autenticación por clave pública

Modificar:

- `PasswordAuthentication no`
-

24. Probar fallo de DNS

Cambiar DNS a uno incorrecto y probar ping a dominio.

ANTES:

```
GNU nano 8.6 /etc/resolv.conf
# Generated by NetworkManager
nameserver 80.58.61.254
nameserver 80.58.61.250
```

DESPUES:

```
GNU nano 8.6 /etc/resolv.conf *
# Generated by NetworkManager
nameserver 1.2.3.4
nameserver 1.2.3.4
```

```
(kali@kali)-[~]
$ ping google.com
ping: google.com: Temporary failure in name resolution
```

25. Analizar tráfico activo por proceso

- `ss -tp`

```
(kali@kali)-[~]
$ ss -tp
State      Recv-Q    Send-Q    Local Address:Port    Peer Address:Port    Process
```

26. trabajo en equipo: Crear una red virtual interna entre varias VMs

- Asignar IPs manualmente y probar ping entre ellas.

Posible solución

✓ Parte A — Preparación (VirtualBox)

1) Crear/usar red “Internal Network”

En **cada VM** (Kali/Parrot/Ubuntu):

1. Apaga la VM si está encendida.
2. Ve a:
VirtualBox → Configuración → Red
3. En **Adaptador 1**:
 - Marca ☒ **Habilitar adaptador de red**
 - Conectado a: **Red interna (Internal Network)**
 - Nombre: escribe el mismo para todos, por ejemplo:
aula-interna
 - Modo promiscuo: **Denegar**
 - Cable conectado: ☒

 **Importantísimo: Todas las VMs deben tener el mismo nombre de red interna (aula-interna).**

✓ Parte B — Direccionamiento IP (manual)

Vamos a usar una red privada tipo:

- Red: 192.168.50.0/24
- Máscara: 255.255.255.0

Ejemplo de reparto por grupos:

- VM1: 192.168.50.10
- VM2: 192.168.50.11
- VM3: 192.168.50.12

(El gateway no es necesario, porque la red es interna y aislada.)

✓ Parte C — Configurar IP dentro de Linux

♦ Opción 1 (recomendada para clase): configuración temporal con ip

En **cada VM**, abre terminal y ejecuta:

1. Ver interfaz de red:

ip a

Localiza la interfaz (normalmente será enp0s3, eth0 o similar).

2. Asignar IP (ejemplo VM1):

```
sudo ip addr add 192.168.50.10/24 dev enp0s3
```

```
sudo ip link set enp0s3 up
```

(En otra VM usa 192.168.50.11, etc.)

3. Verificar:

```
ip a show enp0s3
```

📌 Nota: Si se equivocan de interfaz, no funcionará el ping.

♦ Opción 2: configuración persistente (Ubuntu/Debian/Parrot)

En Ubuntu (netplan) sería algo así:

```
sudo nano /etc/netplan/01-netcfg.yaml
```

Ejemplo:

```
network:
```

```
  version: 2
```

```
  renderer: networkd
```

```
  ethernets:
```

```
    enp0s3:
```

```
      dhcp4: no
```

```
      addresses:
```

```
        - 192.168.50.10/24
```

Aplicar:

```
sudo netplan apply
```

En Kali/Parrot también pueden usar NetworkManager:

```
nmtui
```

✓ Parte D — Pruebas de conectividad

1) Hacer ping entre máquinas

Desde VM1:

```
ping -c 4 192.168.50.11
```

Desde VM2:

```
ping -c 4 192.168.50.10
```

✓ Si responde: red interna OK.

✓ Parte E — Evidencias (para entregar)

Cada grupo debe entregar:

1. Captura de ip a en cada VM.
2. Captura de ping funcionando entre dos máquinas.
3. Breve explicación (5 líneas):
 - qué es Internal Network
 - por qué no hace falta gateway
 - qué rango IP han usado

27. Hacer port forwarding en VirtualBox

Acceder por SSH desde tu PC Host.

28. Crear un script de diagnóstico de red

Debe mostrar:

- IP
- Gateway

- DNS
- Conectividad a Internet

Solución

Script: diag_red.sh

```
#!/bin/bash
```

```
# Script sencillo de diagnóstico de red (FP - Linux)
```

```
echo "=====
```

```
echo "  DIAGNÓSTICO DE RED (FP)  "
```

```
echo "=====
```

```
echo "Equipo: $(hostname)"
```

```
echo "Fecha : $(date)"
```

```
echo
```

```
# 1) IP
```

```
echo "1) IP (dirección del equipo):"
```

```
IP=$(hostname -I | awk '{print $1}')
```

```
if [ -z "$IP" ]; then
```

```
    echo "  IP: No encontrada ❌"
```

```
else
```

```
    echo "  IP: $IP ✅"
```

```
fi
```

```
echo
```

```
# 2) Gateway
```

```
echo "2) Gateway (puerta de enlace):"
```

```
GW=$(ip route | awk '/default/ {print $3}')
```

```
if [ -z "$GW" ]; then
```

```
    echo " Gateway: No hay (normal en Red Interna de VirtualBox) ⚠ "
else
    echo " Gateway: $GW ✅"
fi
echo
```

3) DNS

```
echo "3) DNS (servidores de nombres):"
DNS=$(grep "nameserver" /etc/resolv.conf 2>/dev/null | awk '{print $2}')
if [ -z "$DNS" ]; then
    echo " DNS: No encontrado ❌"
else
    echo " DNS:"
    echo "$DNS" | sed 's/^/ - /'
    echo " ✅"
fi
echo
```

4) Conectividad a Internet

```
echo "4) Conectividad a Internet:"
ping -c 2 -W 1 8.8.8.8 > /dev/null 2>&1
if [ $? -eq 0 ]; then
    echo " Internet: OK (ping a 8.8.8.8) ✅"
else
    echo " Internet: NO hay conexión (ping fallido) ❌"
fi
echo
```



```
echo "Fin del diagnóstico."
```

Cómo usarlo (alumnos)

1. Crear archivo:

```
nano diag_red.sh
```

2. Dar permisos:

```
chmod +x diag_red.sh
```

3. Ejecutar:

```
./diag_red.sh
```

29. Simular caída de red

- `sudo ip link set enp0s3 down`

Y recuperarla:

- `sudo ip link set enp0s3 up`
-

30. Capturar paquetes (Wireshark o tcpdump)

Ejemplo:

- `sudo tcpdump -i enp0s3 -c 20`
-

MINI-PROYECTO DEL MÓDULO 6 EN EQUIPO

Objetivo:

Configurar **un servidor Ubuntu accesible por SSH de forma segura** y documentar todo el proceso.

Tareas:

1. Asignar **IP estática** usando Netplan.
2. Configurar **DNS manuales**.
3. Activar SSH y cambiar puerto.

4. Crear clave pública/privada y desactivar login por contraseña.
 5. Configurar firewall UFW:
 - permitir nuevo puerto SSH
 - denegar puertos innecesarios
 6. Crear script diagnostico_red.sh con:
 - IP actual
 - Gateway
 - DNS
 - Ping a 8.8.8.8
 - Ping a google.com
 7. Probar conexión remota desde otra máquina.
 8. Documentar todo:
 - Archivos modificados
 - Comandos usados
 - Problemas encontrados
 - Solución aplicada
-

Solución posible

Actividad en equipo: Red interna + SSH seguro entre varios puestos (VirtualBox)

Objetivo

Crear una red virtual interna (aislada) donde varias máquinas (Kali/Parrot/Ubuntu) puedan:

- verse por IP (ping),
 - conectarse por **SSH de forma segura**,
 - proteger SSH con claves y firewall,
 - y documentar el proceso.
-

Organización por equipos (ideal 3 alumnos / 3 VMs)

Cada equipo trabaja con **3 VMs**, por ejemplo:

- VM1: Ubuntu Desktop
- VM2: Kali
- VM3: Parrot

✓ Cada VM tendrá:

- IP (dinámica o manual)
 - SSH activo (servidor)
 - claves SSH para acceder a las otras
 - firewall (en Ubuntu/Parrot con UFW; en Kali se puede omitir o usar iptables)
-

0) VirtualBox: Red interna común (obligatorio)

En cada VM:

Adaptador 1 (opcional para Internet)

- NAT

Adaptador 2 (obligatorio para comunicarse)

- Red interna
- Nombre: **aula-interna**

📌 Importante: **el nombre debe ser el mismo en todas.**

1) IP de las máquinas (dos opciones)

✓ **Opción A (recomendada para FP): IP manual temporal (sin Netplan)**

En cada VM poned una IP distinta en la interfaz de red interna (enp0s8, eth1, etc.)

1.1 Identificar interfaz

ip a

1.2 Asignar IP (ejemplos)

En VM1:

```
sudo ip addr add 192.168.50.10/24 dev enp0s8
```

```
sudo ip link set enp0s8 up
```

VM2:

```
sudo ip addr add 192.168.50.11/24 dev enp0s8
```

```
sudo ip link set enp0s8 up
```

VM3:

```
sudo ip addr add 192.168.50.12/24 dev enp0s8
```

```
sudo ip link set enp0s8 up
```

 Esto es temporal (al reiniciar se pierde), pero para clase es perfecto.

2) Comprobar conectividad (ping entre VMs)

Desde VM1:

```
ping -c 3 192.168.50.11
```

```
ping -c 3 192.168.50.12
```

 Si hay respuesta, la red interna funciona.

3) Instalar y activar SSH en TODAS las VMs

Ubuntu / Parrot:

```
sudo apt update
```

```
sudo apt install openssh-server -y
```

```
sudo systemctl enable ssh
```

```
sudo systemctl start ssh
```

Kali:

```
sudo apt update
```

```
sudo apt install openssh-server -y
```

```
sudo systemctl enable ssh
```

```
sudo systemctl start ssh
```

Comprobar:

```
sudo systemctl status ssh --no-pager
```

4) SSH seguro (cambiar puerto + claves + sin contraseña)

4.1 Cambiar puerto SSH (recomendado)

En cada VM editar:

```
sudo nano /etc/ssh/sshd_config
```

Cambiar:

```
#Port 22
```

por:

```
Port 2222
```

Y añadir (o asegurar):

```
PermitRootLogin no
```

```
PasswordAuthentication no
```

```
PubkeyAuthentication yes
```

Reiniciar:

```
sudo systemctl restart ssh
```

Comprobar puerto:

```
ss -tulpn | grep ssh
```

4.2 Crear clave SSH en cada máquina (cada una tendrá su clave)

En cada VM:

```
ssh-keygen -t ed25519
```

(Enter, Enter, Enter)

4.3 Copiar clave a las otras máquinas

Ejemplo: desde VM1 copiar clave a VM2 y VM3:

```
ssh-copy-id -p 2222 usuario@192.168.50.11
```

```
ssh-copy-id -p 2222 usuario@192.168.50.12
```

Y desde VM2 copiar a VM1 y VM3, etc.

 Resultado: cualquiera puede entrar por SSH a cualquiera **sin contraseña**.

5) Firewall (UFW) en Ubuntu / Parrot

En Ubuntu y Parrot:

```
sudo apt install ufw -y
```

```
sudo ufw allow 2222/tcp
```

```
sudo ufw enable
```

```
sudo ufw status verbose
```

✅ Solo queda abierto el puerto 2222.

En Kali no es obligatorio usar UFW (puede hacerse con iptables, pero para FP lo puedes dejar opcional).

6) Script diagnostico_red.sh (en cada VM)

Crear:

```
nano diagnostico_red.sh
```

Pegar:

```
#!/bin/bash
```

```
echo "=====
```

```
echo " DIAGNÓSTICO DE RED (FP)      "
```

```
echo "=====
```

```
echo "Equipo: $(hostname)"
```

```
echo "Fecha : $(date)"
```

```
echo
```

```
echo "IP actual:"
```

```
hostname -I | awk '{print " -", $1}'
```

```
echo
```

```
echo "Gateway:"
```

```
GW=$(ip route | awk '/default/ {print $3}')
```

```
[ -z "$GW" ] && echo " - No hay gateway (red interna)" || echo " - $GW"
echo
```

```
echo "DNS:"
grep nameserver /etc/resolv.conf | awk '{print " -", $2}'
echo
```

```
echo "Ping a 8.8.8.8:"
ping -c 2 -W 1 8.8.8.8 >/dev/null 2>&1
[ $? -eq 0 ] && echo " - OK  " || echo " - FAIL  "
echo
```

```
echo "Ping a google.com:"
ping -c 2 -W 1 google.com >/dev/null 2>&1
[ $? -eq 0 ] && echo " - OK  " || echo " - FAIL  "
```

Permisos:

```
chmod +x diagnostico_red.sh
./diagnostico_red.sh
```

7) Pruebas finales (conexiones SSH cruzadas)

Desde VM1:

```
ssh -p 2222 usuario@192.168.50.11
ssh -p 2222 usuario@192.168.50.12
```

Desde VM2:

```
ssh -p 2222 usuario@192.168.50.10
```

 Debe entrar sin password.

8) Documentación (muy sencilla para FP)

Lo que deben entregar (1 PDF por equipo)

A) Tabla de equipos

VM	SO	IP	Puerto SSH
VM1	Ubuntu	192.168.50.1 0	2222
VM2	Kali	192.168.50.1 1	2222
VM3	Parrot	192.168.50.1 2	2222

B) Archivos modificados

- /etc/ssh/sshd_config

C) Comandos usados (copiar y pegar)

- ip a, ip addr add...
- apt install openssh-server
- ssh-keygen
- ssh-copy-id
- ufw allow 2222/tcp, ufw enable

D) Evidencias (capturas)

- ping entre VMs
- salida de ./diagnostico_red.sh
- conexión SSH realizada

E) Problemas encontrados + solución

Ejemplos:

- No responde ping → red interna mal configurada (nombre distinto)
- SSH no conecta → UFW no permitía 2222
- No entra por clave → no se ejecutó ssh-copy-id